

HDOC – Experiment 17

Question

Write a menu-driven C program using switch-case to perform one of these operations on a positive integer:

- 1. Generate the largest possible number using its digits.**
- 2. Generate the smallest possible number using its digits.**
- 3. Swap the first and last digits.**
- 4. Check whether the number is a palindrome.**

Each option must have:

- 1. A separate user-defined function.**
- 2. void return type.**
- 3. No arguments/parameters.**
- 4. Input taken inside the respective function.**
- 5. Output displayed inside the function.**

❖ Algorithm

Main Algorithm

1. Start.
2. Declare integer variable choice.
3. Display the four menu options.
4. Read choice.
5. Use switch(choice).
6. If choice is 1, call largestNumber().

7. If choice is 2, call `smallestNumber()`.
8. If choice is 3, call `swapFirstLast()`.
9. If choice is 4, call `checkPalindrome()`.
10. Otherwise, display "Invalid choice".
11. Stop.

Option 1: Generate Largest Possible Number

1. Start.
2. Read a positive integer num.
3. Initialize frequency array `count[10]` to zero.
4. Copy num into temp.
5. Repeat while `temp > 0`:
 - Find last digit using `temp % 10`.
 - Increase the frequency of that digit.
 - Remove the last digit using `temp / 10`.
6. Start from digit 9 and move down to 0.
7. For each digit, append it to result as many times as its frequency.
8. Display result.
9. Stop.

Option 2: Generate Smallest Possible Number

1. Start.
2. Read a positive integer num.
3. Initialize `count[10]` to zero.
4. Copy num into temp.
5. Extract each digit and store its frequency in count.

6. Search from digit 1 to 9 for the smallest available non-zero digit.
7. Append this digit first and reduce its frequency by one.
8. Append all zeros.
9. Starting from 1 to 9, append all remaining digits in ascending order.
10. Display the smallest possible number.
11. Stop.

Option 3: Swap First and Last Digits

1. Start.
2. Read a positive integer num.
3. If the number contains only one digit, display it unchanged.
4. Find the last digit using:
$$\text{last} = \text{num} \% 10$$
5. Initialize divisor = 1.
6. Repeatedly multiply divisor by 10 until $\text{num} / \text{divisor} < 10$.
7. Find the first digit:
$$\text{first} = \text{num} / \text{divisor}$$
8. Remove the contribution of the original first and last digits.
9. Put the original last digit in the first position.
10. Put the original first digit in the last position.
11. Display the result.
12. Stop.

Option 4: Check Palindrome Number

1. Start.
2. Read a positive integer num.
3. Store num in temp.
4. Initialize reverse = 0.
5. While temp > 0:-
 - Find the last digit.
 - Add the digit to reverse using:
 - reverse = reverse * 10 + digit
 - Remove the last digit from temp.
6. Compare reverse with num.
7. If both are equal, display "Palindrome number".
8. Otherwise, display "Not a palindrome number".
9. Stop.

❖ Test case table

Test Case	Menu Choice	Input	Expected Output	Actual Output	Result
1	1	42135	54321	Same	Pass
2	1	52231	53221	Same	Pass
3	2	53124	12345	Same	Pass
4	2	42031	10234	Same	Pass
5	3	12345	52341	Same	Pass

6	3	9876	6879	Same	Pass
7	4	12321	Palindrome number	Same	Pass
8	4	12345	Not a palindrome number	Same	Pass

❖ Program code in C

```
#include <stdio.h>

void largestNumber();
void smallestNumber();
void swapFirstLast();
void checkPalindrome();

int main()
{
    int choice;

    printf("MENU\n");
    printf("1. Generate largest possible number\n");
    printf("2. Generate smallest possible number\n");
    printf("3. Swap first and last digits\n");
    printf("4. Check palindrome number\n");

    printf("Enter your choice: ");
    scanf("%d", &choice);
```

```
switch (choice)
{
    case 1:
        largestNumber();
        break;

    case 2:
        smallestNumber();
        break;

    case 3:
        swapFirstLast();
        break;

    case 4:
        checkPalindrome();
        break;

    default:
        printf("Invalid choice\n");
}

return 0;
}
```

```
void largestNumber()
```

```
{  
    long long int num, temp, result = 0;  
    int digit, i;  
    int count[10] = {0};  
  
    printf("Enter a positive integer: ");  
    scanf("%lld", &num);  
  
    temp = num;  
  
    while (temp > 0)  
    {  
        digit = temp % 10;  
        count[digit]++;  
        temp = temp / 10;  
    }  
  
    for (i = 9; i >= 0; i--)  
    {  
        while (count[i] > 0)  
        {  
            result = result * 10 + i;  
            count[i]--;  
        }  
    }  
  
    printf("Largest possible number = %lld\n", result);  
}
```

```
void smallestNumber()
{
    long long int num, temp, result = 0;
    int digit, i;
    int count[10] = {0};

    printf("Enter a positive integer: ");
    scanf("%lld", &num);

    temp = num;

    while (temp > 0)
    {
        digit = temp % 10;
        count[digit]++;
        temp = temp / 10;
    }

    for (i = 1; i <= 9; i++)
    {
        if (count[i] > 0)
        {
            result = i;
            count[i]--;
            break;
        }
    }
```

```
}
```

```
while (count[0] > 0)
```

```
{
```

```
    result = result * 10;
```

```
    count[0]--;
```

```
}
```

```
for (i = 1; i <= 9; i++)
```

```
{
```

```
    while (count[i] > 0)
```

```
    {
```

```
        result = result * 10 + i;
```

```
        count[i]--;
```

```
    }
```

```
}
```

```
printf("Smallest possible number = %lld\n", result);
```

```
}
```

```
void swapFirstLast()
```

```
{
```

```
    long long int num, divisor = 1, result;
```

```
    int first, last;
```

```
    printf("Enter a positive integer: ");
```

```
    scanf("%lld", &num);
```

```
if (num < 10)
{
    printf("Number after swapping = %lld\n", num);
    return;
}

last = num % 10;

while (num / divisor >= 10)
{
    divisor = divisor * 10;
}

first = num / divisor;

result = num;
result = result - first * divisor;
result = result - last;
result = result + last * divisor;
result = result + first;

printf("Number after swapping = %lld\n", result);
}

void checkPalindrome()
{
```

```
long long int num, temp, reverse = 0;

int digit;

printf("Enter a positive integer: ");

scanf("%lld", &num);

temp = num;

while (temp > 0)
{
    digit = temp % 10;
    reverse = reverse * 10 + digit;
    temp = temp / 10;
}

if (reverse == num)
{
    printf("%lld is a palindrome number.\n", num);
}
else
{
    printf("%lld is not a palindrome number.\n", num);
}
}
```

❖ Compile, Execute and Evalute

Test case 1

```
(kali㉿kali)-[~/Desktop/exp17]  
$ ./generate  
MENU  
1. Generate largest possible number  
2. Generate smallest possible number  
3. Swap first and last digits  
4. Check palindrome number  
Enter your choice: 1  
Enter a positive integer: 42135  
Largest possible number = 54321
```

Test case 2

```
(kali㉿kali)-[~/Desktop/exp17]  
$ ./generate  
MENU  
1. Generate largest possible number  
2. Generate smallest possible number  
3. Swap first and last digits  
4. Check palindrome number  
Enter your choice: 1  
Enter a positive integer: 52231  
Largest possible number = 53221
```

Test case 3

```
(kali㉿kali)-[~/Desktop/exp17]  
$ ./generate  
MENU  
1. Generate largest possible number  
2. Generate smallest possible number  
3. Swap first and last digits  
4. Check palindrome number  
Enter your choice: 2  
Enter a positive integer: 53124  
Smallest possible number = 12345
```

Test case 4

```
(kali㉿kali)-[~/Desktop/exp17]  
$ ./generate  
MENU  
1. Generate largest possible number  
2. Generate smallest possible number  
3. Swap first and last digits  
4. Check palindrome number  
Enter your choice: 2  
Enter a positive integer: 42031  
Smallest possible number = 10234
```

Test case 5

```
(kali㉿kali)-[~/Desktop/exp17]  
$ ./generate  
MENU  
1. Generate largest possible number  
2. Generate smallest possible number  
3. Swap first and last digits  
4. Check palindrome number  
Enter your choice: 3  
Enter a positive integer: 12345  
Number after swapping = 52341
```

Test case 6

```
(kali㉿kali)-[~/Desktop/exp17]  
$ ./generate  
MENU  
1. Generate largest possible number  
2. Generate smallest possible number  
3. Swap first and last digits  
4. Check palindrome number  
Enter your choice: 3  
Enter a positive integer: 9876  
Number after swapping = 6879
```

Test case 7

```
(kali㉿kali)-[~/Desktop/exp17]  
$ ./generate  
MENU  
1. Generate largest possible number  
2. Generate smallest possible number  
3. Swap first and last digits  
4. Check palindrome number  
Enter your choice: 4  
Enter a positive integer: 12321  
12321 is a palindrome number.
```

Test case 8

```
(kali㉿kali)-[~/Desktop/exp17]  
$ ./generate  
MENU  
1. Generate largest possible number  
2. Generate smallest possible number  
3. Swap first and last digits  
4. Check palindrome number  
Enter your choice: 4  
Enter a positive integer: 12345  
12345 is not a palindrome number.
```